



Sistema de Monitoreo Ambiental basado en IoT utilizando Raspberry Pi y MQTT

Materia: Ingeniería en Computación II

Carrera: Ingeniería en Computación

Alumno: Marcos Galla

Docentes: Milton Pozzo, Matias Wanzenried

Fecha: Julio 2026

Lugar: Rafaela, Santa Fe

1. INTRODUCCION

El auge del Internet de las Cosas (IoT) y la Industria ha transformado de manera radical la forma en que interactuamos con el entorno físico, permitiendo la automatización, el monitoreo y el control de variables ambientales en tiempo real.

Las arquitecturas tradicionales basadas en HTTP (como las API REST) se vuelven muy ineficientes cuando necesitás transmitir datos en tiempo real. Esto pasa porque el modelo de petición-respuesta genera un montón de overhead innecesario por el tamaño de las cabeceras y la obligación de estar abriendo y cerrando conexiones constantemente.

Para solucionar esto en aplicaciones que requieren flujos continuos o respuestas inmediatas a eventos, la mejor alternativa es usar el protocolo **MQTT**. Al implementar un modelo de publicación/suscripción, es muchísimo más liviano y está optimizado para funcionar sin problemas en redes inestables o de bajo ancho de banda.

El presente trabajo desarrolla un sistema IoT bidireccional basado en arquitectura Edge Computing. La solución está compuesta por un nodo ESP32 que se encarga de la adquisición de datos de temperatura y humedad mediante un sensor DHT11 y del control de un actuador LED, un servidor MQTT Eclipse Mosquitto ejecutado sobre una Raspberry Pi 4 como nodo central de comunicación, y un entorno de orquestación visual Node-RED, el cual asume las tareas de lógica de control y expone una interfaz gráfica (Dashboard) para la supervisión y el envío de comandos en tiempo real. La comunicación entre los componentes se realiza de forma inalámbrica mediante el protocolo MQTT dentro de una red de área local (WLAN).

2. MARCO TEORICO Y ESTADO DEL ARTE

2.1. El Paradigma de Computación en el Borde (*Edge Computing*)

En las arquitecturas tradicionales de IoT, los datos recolectados por los sensores son transmitidos directamente a servidores centralizados en la nube para su procesamiento y almacenamiento. Si bien este enfoque ofrece una gran capacidad de cómputo, introduce problemas críticos como una alta latencia, dependencia de la conectividad a Internet de banda ancha y vulnerabilidades en la privacidad de los datos .

La Computación en el Borde (*Edge Computing*) desplaza el procesamiento de los datos y el almacenamiento hacia la periferia de la red, situándolos físicamente cerca de la fuente de generación de la información. En este proyecto, la adopción de este paradigma permite descentralizar el flujo de datos y operar de forma autónoma dentro de una red de área local (LAN), garantizando tiempos de respuesta mínimos y mayor seguridad en entornos residenciales o industriales controlados.

2.2. Protocolo MQTT y el Modelo Publicación/Suscripción

El protocolo MQTT (Message Queuing Telemetry Transport) es un protocolo de mensajería ligero diseñado específicamente para la comunicación Máquina a Máquina (M2M) en entornos de Internet de las Cosas. Funciona sobre la capa de transporte TCP/IP y se fundamenta en una arquitectura abierta de **Publicación/Suscripción**, la cual desacopla completamente a los clientes que generan información de aquellos que la consumen.

A diferencia del modelo clásico de petición/respuesta de HTTP, los clientes de MQTT no se comunican entre sí directamente, sino que interactúan a través de un agente central denominado **Bróker** [4]. Los flujos de mensajes se organizan mediante etiquetas jerárquicas llamadas **Tópicos** (*Topics*). Un cliente actúa como *publicador* cuando envía un mensaje con un tópico específico al bróker, y actúa como *suscriptor* cuando se registra en el bróker para recibir los mensajes asociados a dicho tópico. Esta estructura permite un intercambio de datos bidireccional, asíncrono y de muy bajo consumo de ancho de banda.

2.3. Herramientas de Software: Mosquitto, Node-RED y Docker

Para la materialización del bróker MQTT, la solución de código abierto de referencia es Eclipse Mosquitto. Debido a su huella de memoria extremadamente reducida, es la herramienta excelente y estándar para entornos embebidos.

En la capa de aplicación y visualización, se terminó por elegir Node-RED, un entorno de desarrollo basado en flujos diseñado originalmente por IBM para la interconexión de hardware y servicios en el ecosistema IoT. Construido sobre Node.js, Node-RED trabaja bajo un modelo asíncrono dirigido por eventos que se acopla de manera nativa al protocolo MQTT. A diferencia de los microframeworks tradicionales basados en código (como Flask), Node-RED minimiza el código base (boilerplate code), acelera el ciclo de prototipado mediante un paradigma low-code y provee un motor de renderizado de dashboards web con una latencia significativamente menor, optimizando la visualización concurrente de telemetría.

Para la orquestación e infraestructura de estos servicios, se incorporó Docker y Docker Compose. Docker es una plataforma de software que permite automatizar el despliegue de aplicaciones dentro de contenedores autónomos. Al encapsular Mosquitto y Node-RED con sus respectivas dependencias y sistemas de archivos aislados, se garantiza la portabilidad absoluta del sistema, independizando la lógica de la aplicación de la configuración particular del sistema operativo anfitrión (Raspberry Pi OS).

2.4. Plataformas de Hardware: ESP32 y Raspberry Pi 4 model B

La heterogeneidad del hardware es una característica intrínseca del IoT. En la capa de adquisición, el **ESP32** es un Sistema en un Chip (SoC) de bajo costo y consumo energético, diseñado por Espressif Systems. Cuenta con un microprocesador Xtensa de doble núcleo, conectividad integrada Wi-Fi y Bluetooth, y una amplia variedad de periféricos de entrada/salida (GPIO). Esto lo convierte en un nodo ideal para interactuar con sensores

ambientales como el **DHT11** —un transductor digital que mide temperatura y humedad mediante un sensor capacitivo y un termistor — y actuar simultáneamente sobre componentes digitales como diodos emisores de luz (LED).

En la capa de coordinación y procesamiento periférico, la **Raspberry Pi 4 Model B** opera como una computadora de placa única de alto rendimiento. Equipada con un procesador ARM Cortex-A72 de 64 bits y memoria RAM de arquitectura LPDDR4, ofrece la capacidad de cómputo y la estabilidad de red (vía Ethernet o Wi-Fi de doble banda) necesarias para ejecutar el sistema operativo embebido basado en Linux donde reside el bróker Mosquitto de forma ininterrumpida.

3. DESARROLLO E IMPLEMENTACION

3.1 Arquitectura general del sistema

Para cumplir con los objetivos planteados se diseñó una arquitectura distribuida compuesta por tres elementos principales: un nodo de adquisición de datos basado en ESP32, un bróker MQTT ejecutándose sobre una Raspberry Pi 4 y un entorno de orquestación y visualización web desarrollado sobre Node-RED.

El ESP32 se encarga de adquirir los datos de temperatura y humedad proporcionados por el sensor DHT11. Una vez obtenidos, estos valores son enviados mediante conexión Wi-Fi al bróker MQTT utilizando un esquema de publicación y suscripción.

La Raspberry Pi 4 cumple la función de servidor de comunicaciones y alojamiento de la capa de aplicación. Sobre ella se desplegó el bróker Mosquitto, encargado de recibir los mensajes enviados por el ESP32 y distribuirlos a los clientes suscritos, así como el entorno Node-RED, el cual consume dicha telemetría de forma asíncrona para actualizar la interfaz gráfica de usuario en tiempo real.

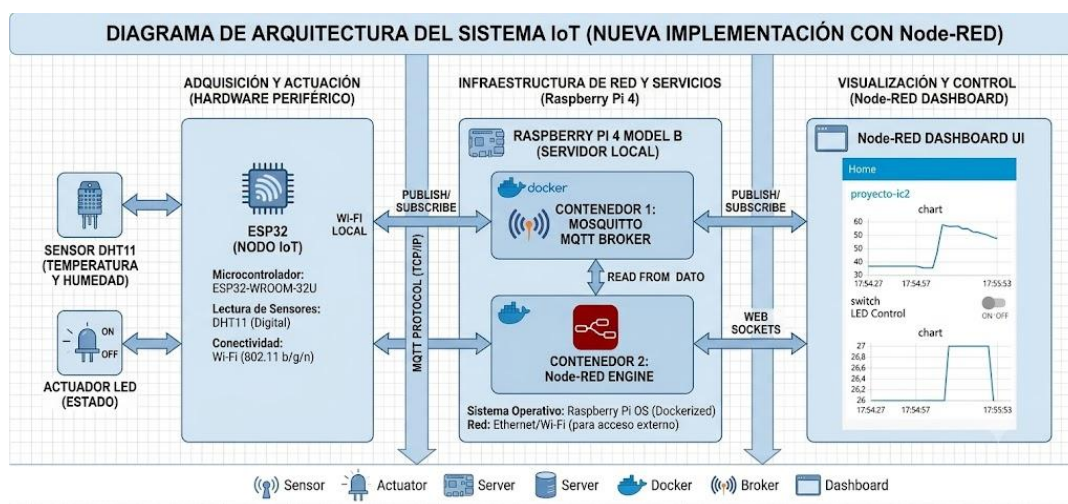


Imagen 1.1

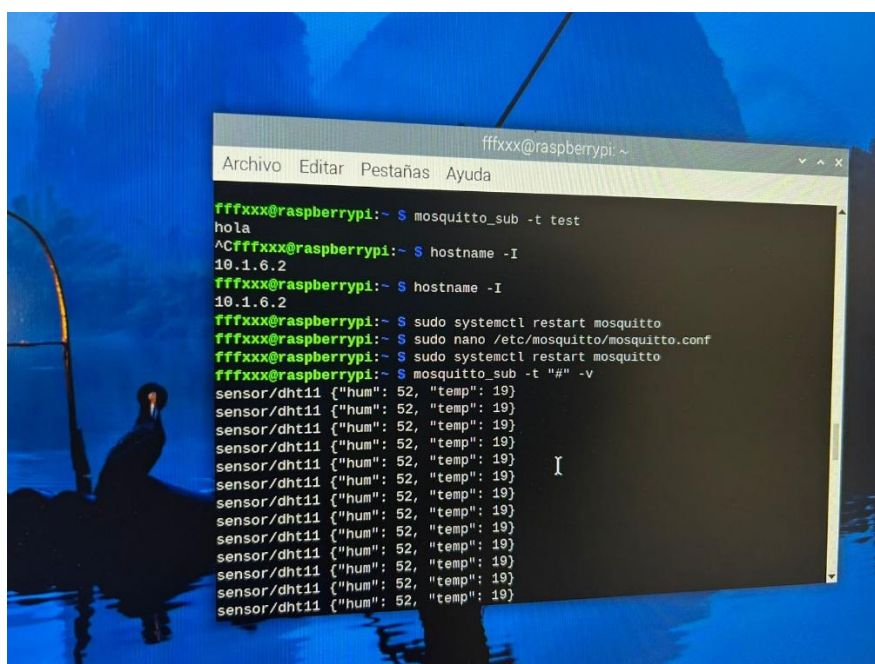
La Imagen 1.1 muestra la arquitectura completa implementada y el flujo de información entre los distintos componentes.

3.2 Configuración del broker MQTT

Con el objetivo de disponer de una infraestructura de comunicación propia, se decidió instalar un broker MQTT local sobre la Raspberry Pi 4 utilizando Mosquitto.

La instalación se realizó sobre Raspberry Pi OS mediante el gestor de paquetes APT. Posteriormente se habilitó el servicio para que se inicie automáticamente junto con el sistema operativo.

Una vez finalizada la instalación, se realizaron pruebas de publicación y suscripción utilizando las herramientas proporcionadas por Mosquitto, verificando el correcto funcionamiento del broker antes de integrar el resto de los componentes del proyecto.

A screenshot of a terminal window on a Raspberry Pi. The window title is 'fffxxx@raspberrypi: ~'. The terminal shows a series of commands and their outputs. The commands include 'mosquitto_sub -t test', 'hostname -I', 'sudo systemctl restart mosquitto', and 'mosquitto_sub -t "#" -v'. The output shows a series of JSON messages from 'sensor/dht11' containing humidity and temperature data. The background of the terminal window shows a blue sky and a silhouette of a person on a boat.

```
fffxxx@raspberrypi:~$ mosquitto_sub -t test
hola
ACfffxxx@raspberrypi:~$ hostname -I
10.1.6.2
fffxxx@raspberrypi:~$ hostname -I
10.1.6.2
fffxxx@raspberrypi:~$ sudo systemctl restart mosquitto
fffxxx@raspberrypi:~$ sudo nano /etc/mosquitto/mosquitto.conf
fffxxx@raspberrypi:~$ sudo systemctl restart mosquitto
fffxxx@raspberrypi:~$ mosquitto_sub -t "#" -v
sensor/dht11 {"hum": 52, "temp": 19}
sensor/dht11 {"hum": 52, "temp": 19}
sensor/dht11 {"hum": 52, "temp": 19}
sensor/dht11 {"hum": 52, "temp": 19}
sensor/dht11 {"hum": 52, "temp": 19}
sensor/dht11 {"hum": 52, "temp": 19}
sensor/dht11 {"hum": 52, "temp": 19}
sensor/dht11 {"hum": 52, "temp": 19}
sensor/dht11 {"hum": 52, "temp": 19}
sensor/dht11 {"hum": 52, "temp": 19}
sensor/dht11 {"hum": 52, "temp": 19}
sensor/dht11 {"hum": 52, "temp": 19}
sensor/dht11 {"hum": 52, "temp": 19}
sensor/dht11 {"hum": 52, "temp": 19}
```

Imagen 1.2

Imagen 1.2. Mensajes MQTT recibidos por el broker Mosquitto ejecutándose sobre Raspberry Pi 4 durante las pruebas de integración.

3.3 Implementación del nodo de adquisición

El nodo de adquisición fue desarrollado utilizando un microcontrolador ESP32 programado en MicroPython.

El sensor DHT11 fue conectado al ESP32 mediante una interfaz digital, permitiendo la lectura periódica de las variables de temperatura y humedad.

El programa desarrollado establece inicialmente una conexión con la red WiFi disponible y posteriormente se conecta al broker MQTT ejecutado sobre la Raspberry Pi.

Una vez establecida la comunicación, el ESP32 realiza mediciones periódicas y publica los datos obtenidos en los tópicos definidos para el sistema. De esta forma, cualquier cliente suscripto puede acceder a la información generada por el sensor.

Además de publicar datos, el microcontrolador permanece suscripto a tópicos de control que permiten recibir comandos enviados desde la interfaz web.

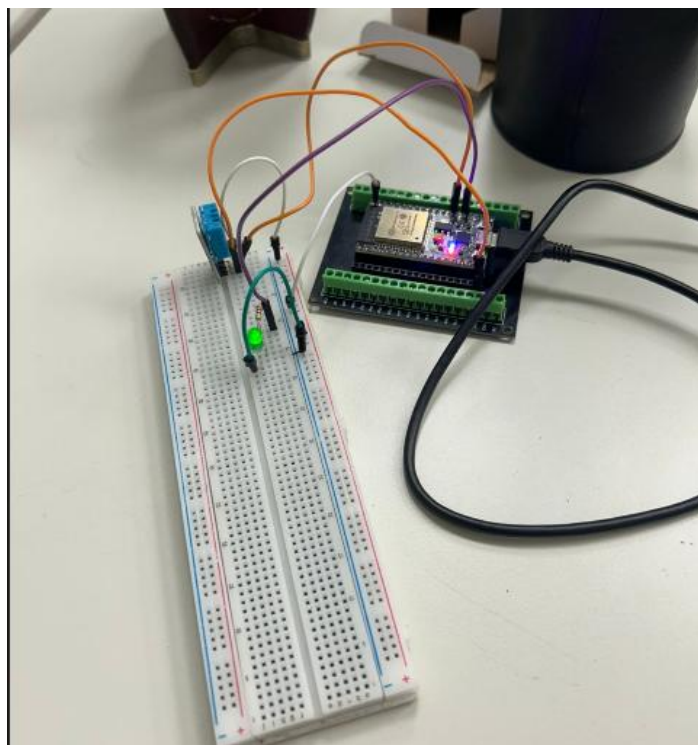


Imagen 1.3

Imagen 1.3. Armado físico del sistema compuesto por ESP32, sensor DHT11, LED indicador y protoboard utilizados durante las pruebas de funcionamiento.

3.4 Desarrollo de la interfaz de supervisión y control en Node-RED

La interfaz de usuario y la lógica de orquestación central del sistema se desarrollaron sobre la plataforma Node-RED, aprovechando su modelo asíncrono basado en flujos e impulsado por eventos. Esta arquitectura permite administrar de forma nativa la concurrencia en la comunicación MQTT mediante un único hilo de ejecución, prescindiendo del uso de hilos secundarios o la gestión manual de bloqueos en el servidor.

Como se detalla en el diagrama de flujos de la herramienta (Imagen 1.4), la aplicación se estructuró mediante tres ramas independientes de procesamiento de mensajes:

- **Procesamiento de temperatura:** Se configuró un nodo de entrada mqtt in enlazado al tópico específico esp32/dht11/temperature. Al recibirse una nueva lectura, el flujo se divide hacia tres nodos: un nodo de depuración (debug 2) para el diagnóstico del flujo en tiempo real, un nodo json encargado de validar la estructura del payload, y un nodo de subrutina programado en JavaScript (function 2). Este último normaliza o extrae la magnitud numérica para inyectarla directamente en un nodo gráfico de tipo chart, encargado de renderizar la tendencia histórica de la variable en el panel web.
- **Procesamiento de humedad:** De manera similar a la temperatura, se configuro un nodo de entrada mqtt in suscrito al tópico esp32/dht11/humidity. El mensaje recibido es procesado simultáneamente por un nodo de diagnóstico (debug 1), un nodo de conversión json, y un nodo de lógica personalizada (function 1), el cual procesa los valores de humedad relativa antes de enviarlos a su correspondiente gráfico temporal (chart) en la interfaz gráfica.
- **Control del LED:** Para garantizar que el estado visual del dashboard coincida con la parte física del hardware, se implementó un lazo de control y estado. Un nodo de entrada mqtt in se encuentra conectado al tópico esp32/led/status para recibir las confirmaciones del estado del LED provenientes del ESP32. Esta señal actúa directamente sobre un nodo de interfaz gráfica tipo switch (conmutador). Cuando el operador interactúa manualmente con este componente en el dashboard, el cambio de estado gatilla de inmediato una publicación a través de un nodo de salida mqtt out hacia el tópico de comando esp32/led/control.

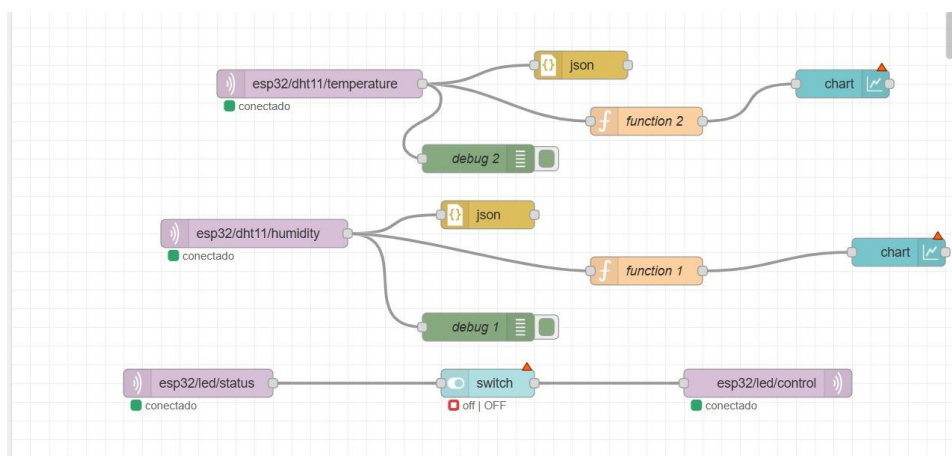


Imagen 1.4. Flujo de programación en NODE RED

La interfaz de usuario web unificada (Imagen 1.5) se agrupa bajo el contenedor denominado proyecto-ic2. Esta interfaz nos permite visualizar en tiempo real los datos a través de dos gráficas temporales (charts) continuas para las variables ambientales y un interruptor central (switch) para el accionamiento remoto y en tiempo real del indicador LED.



Imagen 1.5. Dashboard web ("proyecto-ic2")

3.5 Comunicación bidireccional

Uno de los requisitos principales del proyecto consistía en implementar una comunicación en ambas direcciones entre el sistema microcontrolador y el sistema computador.

La transmisión de datos desde el ESP32 hacia la aplicación web se realiza mediante publicaciones MQTT que contienen los valores obtenidos por el sensor DHT11.

Por otro lado, la interfaz web permite enviar comandos destinados al control de un LED conectado al ESP32. Estos comandos son publicados en tópicos específicos y posteriormente recibidos por el microcontrolador, que ejecuta la acción correspondiente.

Esta arquitectura permitió implementar una comunicación bidireccional completa, donde los dispositivos no solo intercambian información sino que también pueden interactuar entre sí en tiempo real.

3.6 Contenerización y despliegue mediante Docker

Una vez finalizada la implementación funcional del sistema, se incorporó una estrategia de contenerización utilizando Docker con el objetivo de simplificar el despliegue y la ejecución de la aplicación en diferentes entornos. Esta solución permite iniciar todos los servicios necesarios mediante un único comando, reduciendo la dependencia de configuraciones específicas del sistema anfitrión.

La contenerización se realizó mediante Docker Compose, definiendo una arquitectura compuesta por múltiples servicios que se ejecutan de manera coordinada. Entre ellos se incluye un contenedor para el bróker MQTT Mosquitto y un contenedor dedicado para el entorno Node-RED.

Para lograr la integración eficiente entre los servicios dentro de la red virtual de Docker, se parametrizó la dirección del bróker MQTT en los nodos de entrada y salida de Node-RED. Esto permite que la aplicación mantenga la portabilidad absoluta y funcione de manera idéntica tanto en entornos aislados de desarrollo como en la infraestructura física de producción.

Durante las pruebas realizadas se verificó que la totalidad del sistema podía iniciarse correctamente mediante la ejecución del comando Docker Compose, estableciendo automáticamente las conexiones entre los distintos servicios y permitiendo la visualización de datos en tiempo real desde la interfaz web.

3.7 Integración y pruebas del sistema

Una vez implementados los distintos módulos, se realizaron pruebas integrales para verificar el funcionamiento del sistema completo.

En primer lugar, se comprobó la correcta adquisición de datos por parte del ESP32 y su publicación en el broker MQTT. Posteriormente se verificó la recepción de dichos mensajes por parte de la aplicación web.

Finalmente se realizaron pruebas de control remoto utilizando los botones de la interfaz para encender y apagar el LED conectado al microcontrolador.

Los resultados obtenidos demostraron que la arquitectura implementada permite transmitir información de manera confiable entre los distintos componentes y cumplir con los objetivos establecidos para el proyecto.

4.CONCLUSIONES

El desarrollo de este proyecto permitió implementar una solución basada en tecnologías de Internet de las Cosas (IoT) capaz de adquirir, transmitir y visualizar información ambiental en tiempo real. A través de la integración de un microcontrolador ESP32, un sensor DHT11, un bróker MQTT ejecutado sobre una Raspberry Pi 4 y una interfaz de control/supervisión desarrollada sobre Node-RED, fue posible construir una arquitectura distribuida eficiente y de baja latencia que cumple con los objetivos planteados inicialmente.

Durante el desarrollo se aplicaron conocimientos relacionados con programación de microcontroladores, protocolos de comunicación, redes, sistemas embebidos y desarrollo web. La utilización de MQTT permitió establecer una comunicación eficiente entre los

distintos componentes del sistema mediante un esquema de publicación y suscripción, simplificando el intercambio de información y favoreciendo la escalabilidad de la solución.

Las pruebas realizadas demostraron el correcto funcionamiento de la adquisición de datos ambientales, la transmisión de información a través del broker MQTT y la visualización de los valores obtenidos mediante una interfaz accesible para el usuario. Asimismo, se verificó la implementación de una comunicación bidireccional mediante el control remoto de un LED conectado al ESP32 y la modificación del intervalo de muestreo desde la aplicación web.

Uno de los aspectos más importantes del proyecto fue la integración de múltiples tecnologías dentro de una misma solución funcional. Esto permitió comprender de manera práctica cómo interactúan los diferentes elementos que conforman una arquitectura IoT moderna, desde la adquisición de datos en el dispositivo embebido hasta la presentación de la información al usuario final.

En términos generales, los resultados obtenidos permiten concluir que la arquitectura propuesta constituye una alternativa viable para aplicaciones de monitoreo y control remoto de bajo costo, cumpliendo satisfactoriamente con los requisitos establecidos para el proyecto.

5. TRABAJOS FUTUROS Y MEJORAS PROPUESTAS

Si bien el sistema desarrollado cumplió con los objetivos planteados, existen diversas mejoras que podrían incorporarse en futuras versiones para ampliar sus capacidades y acercarlo a escenarios de aplicación reales.

Una de las posibles mejoras consiste en implementar mecanismos de almacenamiento histórico de los datos adquiridos, permitiendo generar registros y realizar análisis posteriores sobre la información obtenida por los sensores.

También podría incorporarse un sistema de autenticación y control de acceso para incrementar la seguridad de las comunicaciones y restringir el acceso a usuarios autorizados.

Otra mejora interesante sería la incorporación de nuevos sensores, tales como sensores de calidad de aire, luminosidad o presión atmosférica, permitiendo ampliar la cantidad de variables monitoreadas por el sistema.

Desde el punto de vista energético, podría implementarse el uso de modos de bajo consumo del ESP32, reduciendo el gasto energético durante los períodos de inactividad y aumentando la autonomía del dispositivo en aplicaciones alimentadas por baterías.

Asimismo, la aplicación podría evolucionar hacia una arquitectura accesible desde Internet mediante servicios en la nube, permitiendo supervisar y controlar el sistema desde cualquier ubicación sin depender de una red local.

Finalmente, la incorporación de gráficos históricos, alertas automáticas y notificaciones ante condiciones específicas permitiría mejorar la experiencia del usuario y aumentar las posibilidades de aplicación práctica del sistema desarrollado.

6. BIBLIOGRAFÍAS Y REFERENCIAS

- 1- Banks, A., & Gupta, R. (2014). *MQTT Version 3.1.1*. OASIS Standard. Disponible en: <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/>
- 2- Espressif Systems. (2024). *ESP32 Series Datasheet*. Disponible en: <https://www.espressif.com>
- 3- Raspberry Pi Foundation. (2024). *Raspberry Pi Documentation*. Disponible en: <https://www.raspberrypi.com/documentation/>
- 4- Eclipse Foundation. (2024). *Eclipse Mosquitto Documentation*. Disponible en: <https://mosquitto.org/documentation/>
- 5- MicroPython Project. (2024). *MicroPython Documentation*. Disponible en: <https://docs.micropython.org/>
- 6- Aosong Electronics. (2018). *DHT11 Temperature and Humidity Sensor Datasheet*. Disponible en: <https://www.aosong.com>

